資料結構
Data Structure

Lab 07

姓名: 夏宇柔
學號: 113840005

Lab07-Ex1.
Complete the following parts in the sample code: **- precedence** **- InfixToPostfix** **Add comments to the program to clearly describe its actions.** **Draw a table showing the step-by-step process of converting each infix expression to postfix (following the method presented in the lecture notes.).**
Code

```cpp
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;  // 存儲字符 (運算子或括號)
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push 操作：將元素放入堆疊
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;       // 存入數據
        newNode->next = top;      // 讓新節點指向目前的頂端
        top = newNode;            // 更新頂端指標
    }

    // Pop 操作：移除並回傳頂端元素
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
        char ch = top->data;        // 取得頂端元素
        Node* temp = top;           // 暫存頂端節點
        top = top->next;            // 移動頂端指標
        delete temp;                // 釋放記憶體
        return ch;                  // 回傳彈出的元素
    }

    // Peek 操作：取得頂端元素但不移除
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};
```

```
// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
    if (op == '+' || op == '-') return 1; // set precedence of + and - to 1
    if (op == '/' || op == '*') return 2; // precedence of * and / to 2
    return 0; // precedence of anything else to 0
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    int j = 0;
    Stack s; // create stack
    for (int i = 0; infix[i] != '\0'; i++) { // loop through all of infix
        char ch = infix[i]; // get current character
        int _ascii = int(ch); // get current character's decimal value
        if ((_ascii <= 57 && _ascii >= 48) || (_ascii <= 90 && _ascii >= 65)
|| (_ascii <= 122 && _ascii >= 97)) { // determine if current character is
0-9, A-Z or a-z
            postfix[j] = ch; // put directly into postfix
            j++;
        }else if (ch == '(') { // put ( directly into stack
            s.push(ch);
        }else if (ch == ')') {
            while (!s.isEmpty() && s.peek() != '(') { // pop until (
                postfix[j] = s.pop(); // put popped into postfix
                j++;
            }
            s.pop(); // pop ( without putting into postfix
        }else { //  process +-*/
            while (!s.isEmpty() && precedence(s.peek()) >= precedence(ch)) {
// check precedence
                postfix[j] = s.pop(); // pop into postfix
                j++;
            }
            s.push(ch); // push current into stack
        }
    }
    // pop all remaining characters in stack
    while (!s.isEmpty()) {
        postfix[j] = s.pop();
        j++;
    }
    postfix[j] = '\0'; // end postfix
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

    return 0;
}
```

Discussion Section

Test expressions process tables:

Test 1: (A+B)*D+E/(F+A*D)+C to AB+D*EFAD*+/+C+

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
(	(					0	
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	+	*	4	AB+D*EFA
D	+	/	(	+	*	4	AB+D*EFAD
)	+	/				1	AB+D*EFAD*+
+	+					2	AB+D*EFAD*+/+
C	+					0	AB+D*EFAD*+/+C
<eos>						-1	AB+D*EFAD*+/+C+

Test 2: A+B*(C*(D+E)) to ABCDE+**+

Token	Stack						Top	Output
	[0]	[1]	[2]	[3]	[4]	[5]		
A							-1	A
+	+						0	A
B	+						0	AB

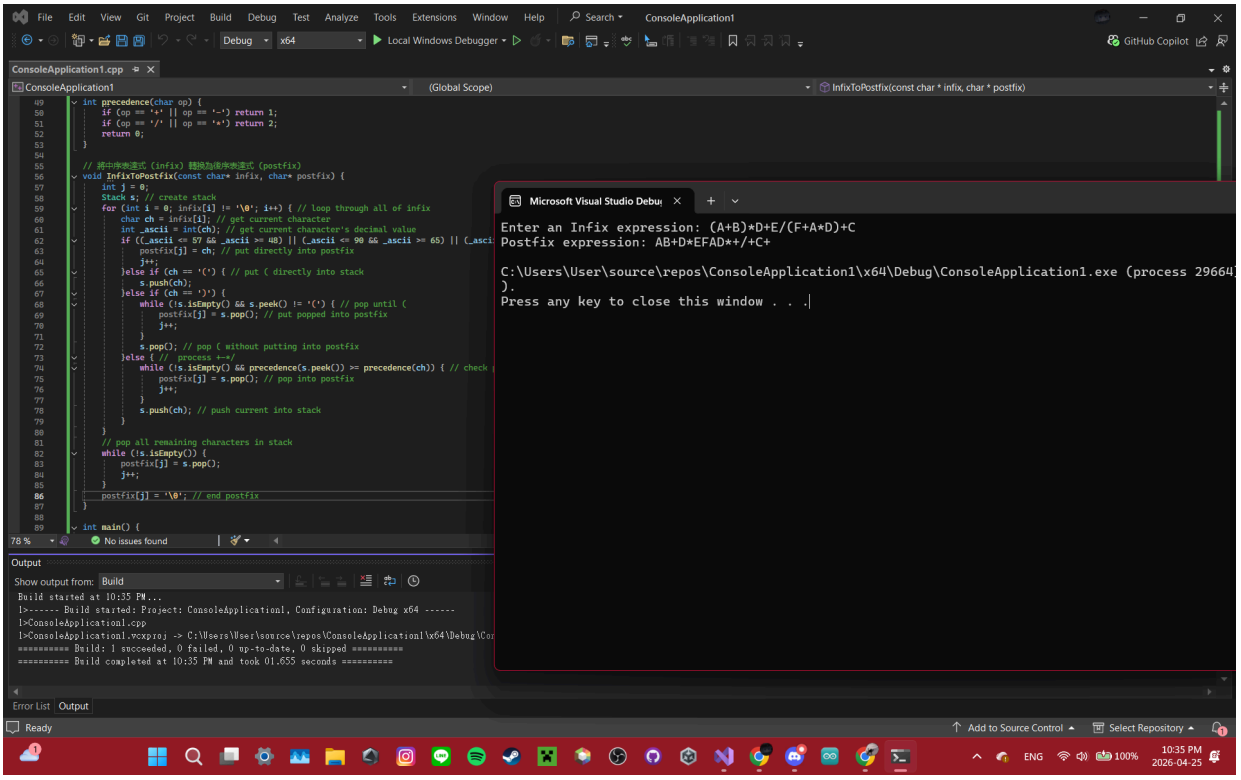
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD
E	+	*	(	*	(	+	5	ABCDE
)	+	*	(	*			3	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+**+

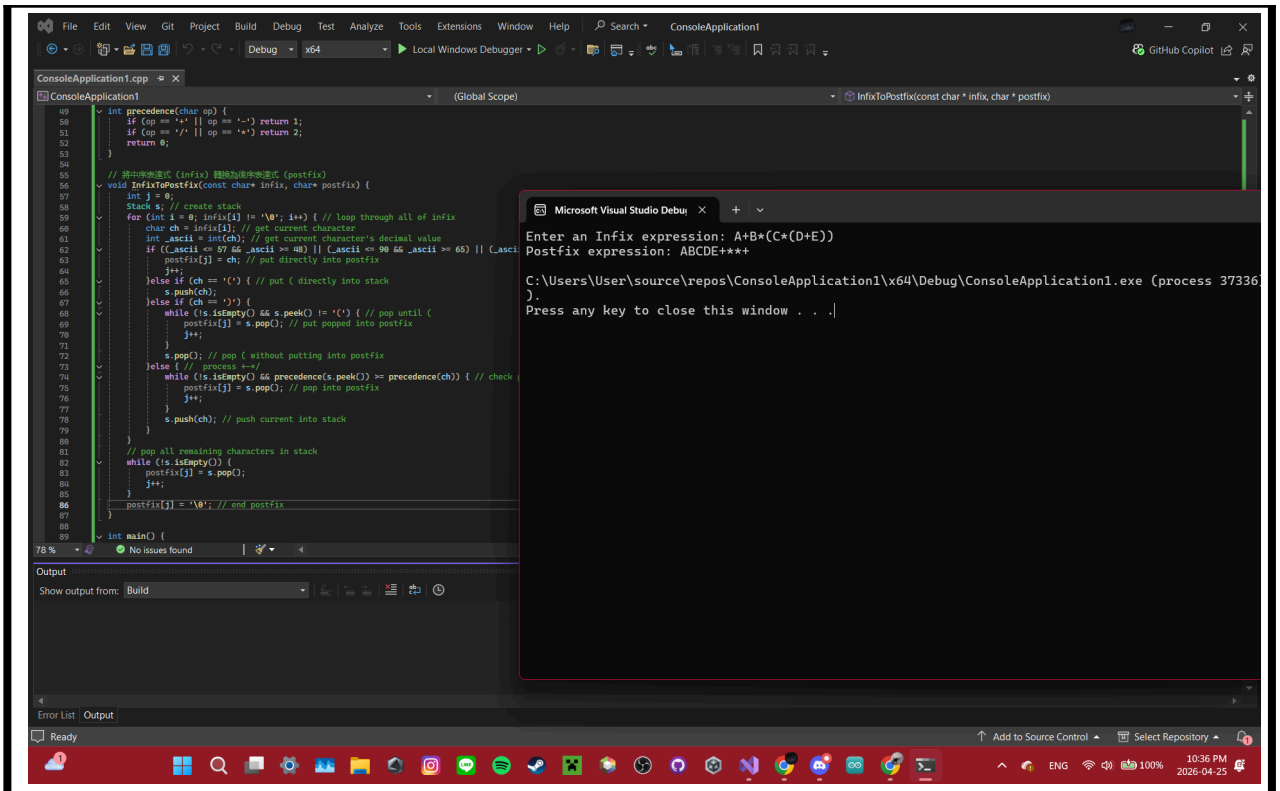
Test 3: 3+1*2+(4+5*6)/(9-7) to 312*+456*+97-/+

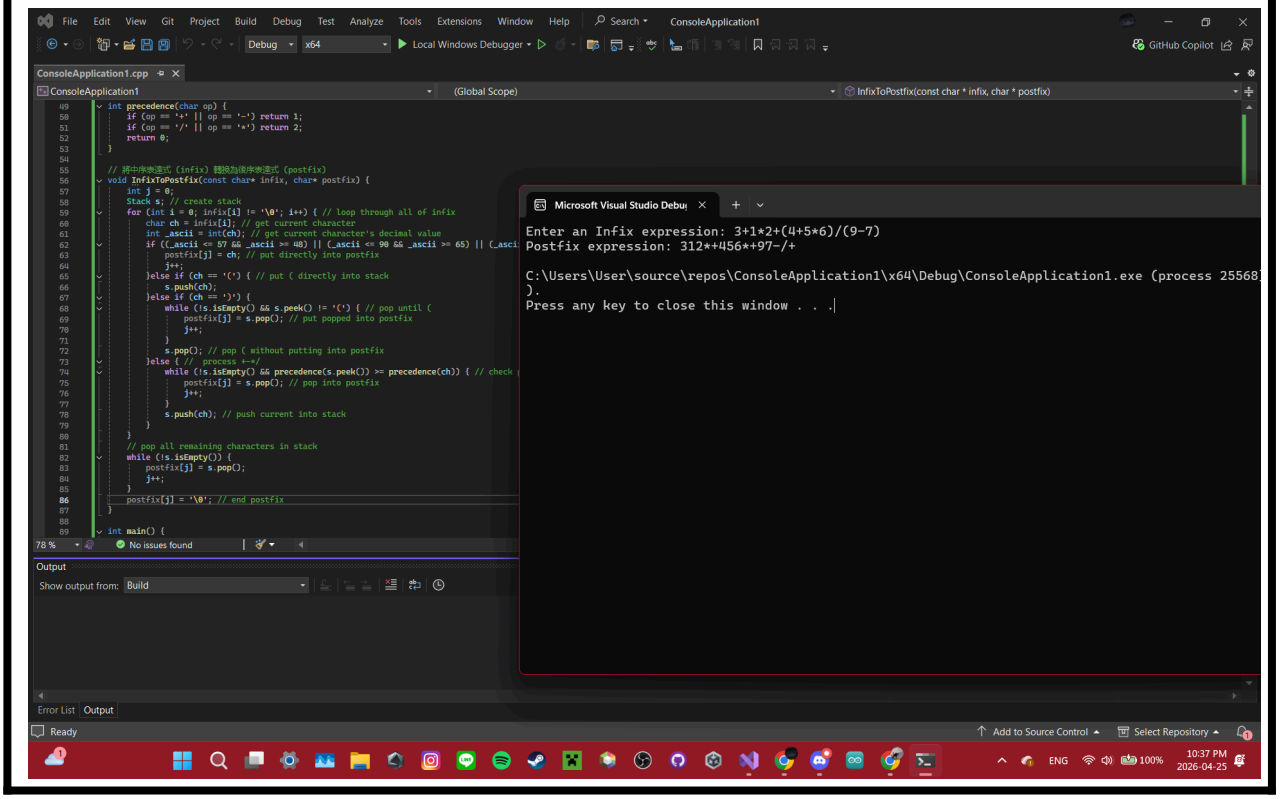
Token	Stack				Top	Output
	[0]	[1]	[2]	[3]		
3					-1	3
+	+				0	3
1	+				0	31
*	+	*			1	31
2	+	*			1	312
+	+				0	312*+
(	+	(			1	312*+
4	+	(			1	312*+4
+	+	(	+		2	312*+4
5	+	(	+		2	312*+45
*	+	(	+	*	3	312*+45
6	+	(	+	*	3	312*+456
)	+				0	312*+456*+
/	+	/			1	312*+456*+
(	+	/	(		2	312*+456*+

9	+	/	(		2	312*+456*+9
-	+	/	(	-	3	312*+456*+9
7	+	/	(	-	3	312*+456*+97
)	+	/			1	312*+456*+97-
<eos>					-1	312*+456*+97-/+

Screenshots:




ConsoleApplication1.cpp
(Global Scope)
InfixToPostfix(const char * infix, char * postfix)
int precedence(char op) {
if (op == '+' || op == '-') return 1;
if (op == '/' || op == '*') return 2;
return 0;
}
// 将中缀表达式 (infix) 转换为逆序表达式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
int j = 0;
Stack s; // create stack
for (int i = 0; infix[i] != '\0'; i++) { // loop through all of infix
char ch = infix[i]; // get current character
int _ascii = int(ch); // get current character's decimal value
if ((_ascii <= 57 && _ascii >= 48) || (_ascii <= 90 && _ascii >= 65) || (_asci
postfix[j] = ch; // put directly into postfix
j++;
}else if (ch == '(') { // put (directly into stack
s.push(ch);
}else if (ch == ')') {
while (!s.isEmpty() && s.peek() != '(') { // pop until (
postfix[j] = s.pop(); // put popped into postfix
j++;
}
s.pop(); // pop (without putting into postfix
}else { // process +-*/
while (!s.isEmpty() && precedence(s.peek()) >= precedence(ch)) { // check
postfix[j] = s.pop(); // pop into postfix
j++;
}
s.push(ch); // push current into stack
}
}
// pop all remaining characters in stack
while (!s.isEmpty()) {
postfix[j] = s.pop();
j++;
}
postfix[j] = '\0'; // end postfix
}
int main() {
No issues found
Output
Show output from: Build
Microsoft Visual Studio Debu
Enter an Infix expression: A+B*(C*(D+E))
Postfix expression: ABCDE+**+
C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 37336
).
Press any key to close this window . . .
Ready
Add to Source Control
Select Repository
10:36 PM
2026-04-25


ConsoleApplication1.cpp
(Global Scope)
InfixToPostfix(const char * infix, char * postfix)
int precedence(char op) {
if (op == '+' || op == '-') return 1;
if (op == '/' || op == '*') return 2;
return 0;
}
// 将中缀表达式 (infix) 转换为逆序表达式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
int j = 0;
Stack s; // create stack
for (int i = 0; infix[i] != '\0'; i++) { // loop through all of infix
char ch = infix[i]; // get current character
int _ascii = int(ch); // get current character's decimal value
if ((_ascii <= 57 && _ascii >= 48) || (_ascii <= 90 && _ascii >= 65) || (_asci
postfix[j] = ch; // put directly into postfix
j++;
}else if (ch == '(') { // put (directly into stack
s.push(ch);
}else if (ch == ')') {
while (!s.isEmpty() && s.peek() != '(') { // pop until (
postfix[j] = s.pop(); // put popped into postfix
j++;
}
s.pop(); // pop (without putting into postfix
}else { // process +-*/
while (!s.isEmpty() && precedence(s.peek()) >= precedence(ch)) { // check
postfix[j] = s.pop(); // pop into postfix
j++;
}
s.push(ch); // push current into stack
}
}
// pop all remaining characters in stack
while (!s.isEmpty()) {
postfix[j] = s.pop();
j++;
}
postfix[j] = '\0'; // end postfix
}
int main() {
No issues found
Output
Show output from: Build
Microsoft Visual Studio Debu
Enter an Infix expression: 3+1*2+(4+5*6)/(9-7)
Postfix expression: 312**456*+97-/+
C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 25568
).
Press any key to close this window . . .
Ready
Add to Source Control
Select Repository
10:37 PM
2026-04-25